



Testing AI workflows locally with Testcontainers

HELLO!

About **me**



Anna Chernyshova

Sr. Solutions Engineer @ Docker

Cat or Dog? Dog!
Coffee or tea? Coffee!
Tabs or spaces? Claude Code.

 <https://www.linkedin.com/in/chernyshovaanna/>

 https://x.com/AnnaChe_

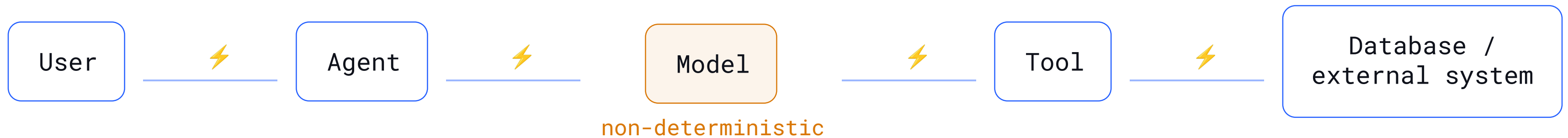
AGENDA

The next 30 minutes

- 01 Why a correct answer from the agent isn't enough
- 02 The toolbox: Testcontainers, Microcks, MCP, Toxiproxy
- 03 Live demos: happy path, slow model and MCP, a lying model
- 04 Q&A

THE CHALLENGE

Non-deterministic component in a distributed system



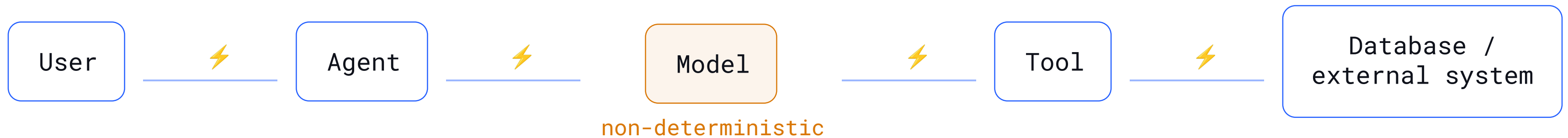
⚡ Every connection is a failure boundary and one box **changes its mind on every run**.

"Non-deterministic? No problem! You can test it!" Eric Deandrea & Oleg Šelajev
youtube.com/watch?v=C7-XS1L1FNs

THE CHALLENGE

Non-deterministic component in a distributed system

Make model replies **deterministic** to validate the agentic workflow with its MCP calls, and to emulate model failure on demand.



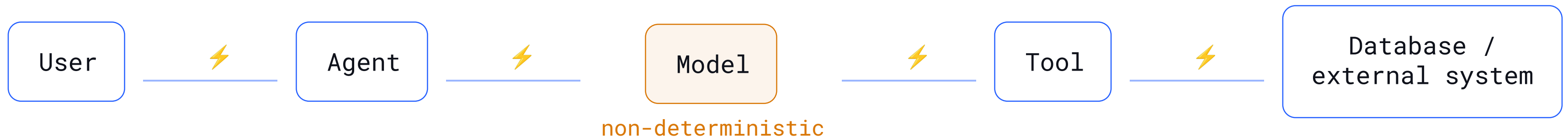
⚡ Every connection is a failure boundary and one box **changes its mind on every run**.

"Non-deterministic? No problem! You can test it!" Eric Deandrea & Oleg Šelajev
youtube.com/watch?v=C7-XS1L1FNs

THE CHALLENGE

Non-deterministic component in a distributed system

Make model replies **deterministic** to validate the agentic workflow with its MCP calls, and to emulate model failure on demand.



⚡ Every connection is a failure boundary and one box **changes its mind on every run**.

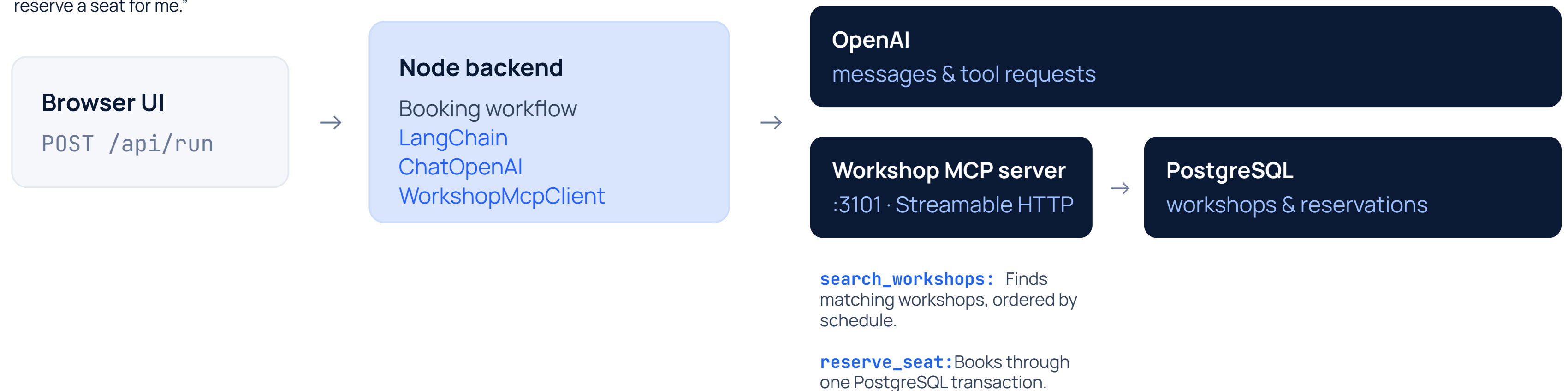
Introduce **controlled delays** of the Model response and Tool call to verify edge cases

"Non-deterministic? No problem! You can test it!" Eric Deandrea & Oleg Šelajev
youtube.com/watch?v=C7-XS1L1FNs

THE DEMO PROJECT

Meet the **Workshop booking assistant**

“Find an afternoon workshop about testing AI applications and reserve a seat for me.”



LangSmith tracing — one trace per run with model messages, tool calls and MCP round-trips.

TESTING APPROACH

Bring the whole workflow **to the laptop**

1

Run every piece locally

Disposable containers created and destroyed per test run

2

Script only the model

The one nondeterministic part, everything else stays real

3

Assert on calls and state

Facts, not wording: tool calls, protocols, database rows

Test dependencies **as code**

Testcontainers is an open source library for providing ephemeral, lightweight instances of test dependencies.

```
const postgres = await new PostgreSQLContainer("postgres:17.6-alpine")
    .withDatabase("last_seat")
    .withUsername("last_seat")
    .withPassword("last_seat")
    .withNetwork(network)
    .withNetworkAliases("postgres")
    .start()
```



Real tools, **containerized**

With Testcontainers you can run MCP Server from a local DOckerfile as a **container**.

```
const mcp = await new GenericContainer.fromDockerfile(projectRoot, "Dockerfile.mcp")
    .withCache(true)
    .build(IMAGES.mcp, { deleteOnExit: false });

mcpContainer = await mcpImage.withEnvironment({DATABASE_URL: internalDatabaseUrl})
    .withExposedPorts(WORKSHOP_MCP_PORT)
    .withNetwork(network)
    .withNetworkAliases("workshop-mcp-server")
    .withWaitStrategy(
        Wait.forHttp("/health", WORKSHOP_MCP_PORT).forStatusCode(200)
    )
    .start();
```



A model **you can script**

Microcks is an open source mock server that turns API specs into live, scriptable endpoints. That includes an OpenAI-compatible model.

```
const microcks = await new MicrocksContainer("docker.io/microcks/microcks-uber:1.14.0")
    .withMainArtifacts([
        path.join(projectRoot, "microcks/openai-chat-completions.yaml")
    ])
    .withNetwork(network)
    .withNetworkAliases("microcks")
    .start()
```



Failure on demand

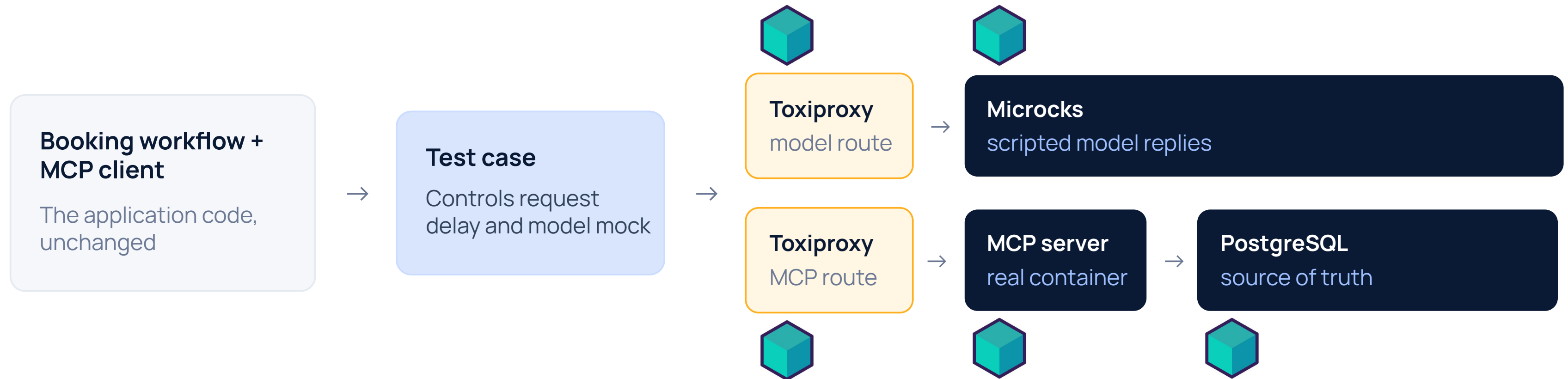
Toxiproxy is a programmable TCP proxy built for failure injection. Latency, timeouts and disconnects become test inputs.

```
toxiproxy = await new ToxiProxyContainer("ghcr.io/shopify/toxiproxy:2.12.0")
    .withNetwork(network)
    .withNetworkAliases("toxiproxy")
    .start();

const modelProxy = await toxiproxy.createProxy({
  name: "model-api",
  upstream: "microcks:8080"
});

const mcpProxy = await toxiproxy.createProxy({
  name: "workshop-mcp",
  upstream: `workshop-mcp-server:${WORKSHOP_MCP_PORT}`
});
```

A disposable local lab for the whole workflow



Vitest asserts rows, attempts and remaining seats – in PostgreSQL.

TEST SCENARIOS

Three ways to break the booking

Happy path

Slow model

Slow MCP

Lying model

SCENARIO 01

Happy path: **exactly one seat**

The scripted model searches, finds “**Testing Agentic Workflows**” and reserves the last seat.

The test pins the **exact tool sequence and arguments** — any drift in the workflow shows up immediately.

The transcript can say anything. The test asserts rows and remaining capacity in PostgreSQL.

```
const outcome = await runScenario("happy");
const state = await getDemoState(pool);

expect(outcome.status).toBe("confirmed");
expect(outcome.attempts).toEqual(
  { model: 3, search: 1, reserve: 1 });
expect(toolNames).toEqual(
  ["search_workshops", "reserve_seat"]);

// the source of record:
expect(state.reservations).toHaveLength(1);
expect(remainingSeats(state)).toBe(0);
```

```
npm run test:integration:happy
```

SCENARIO 02

The model is **slow**

Toxiproxy adds **900 ms latency** on the model route against a **250 ms client timeout**.

Slow first reply

The initial response never arrives in time. The workflow fails fast and bounded — no tools run, no seat is booked. And a late reply cannot resume the ended loop.

```
0 tool calls · 0 reservations elapsed < 1.5 s
```

Delayed final explanation

The booking succeeds, then the model's closing summary times out. The reservation stays confirmed — the workflow falls back to its own confirmation.

```
confirmed · 1 row fallback.confirmation
```

```
const outcome = await runScenario("model_timeout", { toxiproxy: { modelLatencyMs: 900 }, config: {  
  modelTimeoutMs: 250 } });  
expect(outcome.attempts).toEqual({ model: 1, search: 0, reserve: 0, reconcile: 0 });  
expect(state.reservations).toHaveLength(0);  
expect(outcome.message).toContain("No booking action was taken");
```

```
npm run test:integration:model-timeouts
```

SCENARIO 03

MCP tools are **slow**

Same trick on the MCP route: **800 ms latency**, **250 ms timeout**.

Slow search

search_workshops times out. The agent reports “availability temporarily unavailable”, it does not invent workshops or book anything.

explicit failure · 0 reservations nothing fabricated

Reservation timeout

The write commits, but the reply is late. The workflow reconciles with **get_reservation** using the same request ID, no second write, no double booking.

confirmed · 1 row reconcile: 1 · seat booked once

```
const outcome = await runScenario("search_timeout", { toxiproxy: { tool: "reserve_seat", mcpLatencyMs: 800 },
config: { mcpTimeoutMs: 250 } });
expect(outcome.attempts).toEqual({ model: 1, search: 1, reserve: 0, reconcile: 0 });
expect(outcome.message).toContain("availability is temporarily unavailable");
expect(state.reservations).toHaveLength(0);
```

`npm run test:integration:mcp-timeouts`

SCENARIO 04

The **lying** model

“Your seat is booked!”

A model can confirm a booking that never happened, hallucinate a workshop, or invent IDs with total confidence.

With Microcks, the lie is **scripted on purpose** : a hallucinated workshop ID, an invented request ID, a cheerful confirmation with no write behind it.

```
Nonsense instead of tool call:
value:
  id: chatcmpl-nonsense-001
  object: chat.completion
  created: 1789488000
  model: microcks-gpt
  choices:
    - index: 0
      message:
        role: assistant
        content: Bananas can fly. The moon is made of workshop tickets.
```

```
// assert the source of record:
expect(outcome.status).toBe("failed");
expect(outcome.message).toBe("The model did not call search_workshops, so no availability was invented and no reservation was attempted.");
expect(outcome.reservation).toBeUndefined();
expect(outcome.attempts).toEqual({ model: 1, search: 0, reserve: 0, reconcile: 0 });
expect(outcome.modelToolCalls).toHaveLength(0);
```

```
npm run test:integration:nonsense-model
```

Where these tests **fit**

01

Local deterministic workflow tests

Protocols, state, and failure edge cases.

02

Model evaluations

Model behavior and answer quality.

03

Real-provider contract tests

A small number, run against the hosted model.

Microcks controls deterministic behavior; it does not prove hosted-model quality.

YOU'RE PROBABLY THINKING...

Fair questions

→ **Does this replace evals?**

No. Evals judge answer quality. These tests guard protocols, state, and failure handling.

→ **Isn't this too slow for CI?**

Containers boot in seconds, run in parallel, and need no shared environment.

→ **What about real model quality?**

Keep a small contract-test suite against the hosted model. Alongside, not instead.

Final summary

1 **Scripted model** replies make the whole agentic workflow deterministic under test.

2 **Toxiproxy** turns timeouts, late writes and races into repeatable scenarios.

3 **Real-model** tool selection and answer quality still need separate evaluations.